

Module 30: Operator Overloading II & Friends

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

Module 13 covered `+`, `==` and `<<` (with `friend` as part of the `<<` recipe). This module completes the toolkit for building value types that behave like built-in numbers. Students learn what `friend` really grants and when to use it, why symmetric operators must be non-members, the standard pattern of `+` written in terms of `+=`, prefix and postfix increment, `const` and non-`const` subscript with bounds checking, function objects and their relationship to lambdas, `operator>>` that fails cleanly, explicit conversion operators including explicit `operator bool`, and C++20's spaceship operator `<=>`. Judgement about which operators to provide runs through every step.

Suggested Duration: 90–105 minutes

Learning Objectives

By the end of this module, students will be able to:

- Use `friend` functions and `friend` classes deliberately, and explain when not to.
- Choose member or non-member operators, and make symmetric operators work with either operand first.
- Write compound assignment operators and implement binary operators in terms of them.
- Overload prefix and postfix `++/--`, and `const` and non-`const` `operator[]`.
- Write function objects and use them with STL algorithms, relating them to lambdas.
- Overload `operator>>` so that bad input fails the stream and leaves the object unchanged.
- Write explicit conversion operators, and use defaulted and hand-written `<=>` (C++20).

Prerequisites

- Part 2 Module 13 (operator overloading) and Module 17 (STL algorithms and lambdas).
- Module 21 (explicit, `const` correctness).

Materials & Instructor Prep

- 30-operator-overloading-ii/examples/ 01 to 08; 08_spaceship.cpp needs `-std=c++20` - check student compilers support it.
- examples/05_functor.cpp and a lambda version side by side.

Key Vocabulary

Term	Definition
friend	A declaration inside a class granting a specific outside function or class access to its private members.
Compound assignment	Operators such as += and -= that modify the left operand and return it by reference.
Prefix / postfix	++x changes then returns the new value; x++ returns the old value. Postfix takes a dummy int parameter.
Function object (functor)	An object whose class overloads operator(), so it can be called like a function while carrying data.
Conversion operator	A member such as explicit operator double() const that converts an object to another type.
Three-way comparison (<=>)	C++20's 'spaceship' operator, answering less/equal/greater in one call; defaulting it gives all comparisons.

Lesson Outline & Instructor Talking Points

1. friend, and member vs non-member (15 min)

- Run examples/01_friend.cpp. Stress that friendship is granted by the class, declared next to the members it uses, and not inherited.
- Write `v * 2.0` and `2.0 * v` on the board and ask why the second can't be a member of `Vector2D`.

2. += first, then + (15 min)

- Run examples/02_compound_and_binary.cpp. Point out that the arithmetic lives only in +=, and + takes its left operand by value.
- Show chaining with `(a += b) += c` and connect the reference return to built-in types' behaviour.

3. ++/-- and [] (20 min)

- Run examples/03_increment.cpp after predictions. Explain why postfix must copy, and why ++i is often preferred.
- Run examples/04_subscript.cpp. Remove the const version and show `average(const Gradebook&)` failing to compile.

4. Functors and >> (20 min)

- Run examples/05_functor.cpp. Show that `IsAbove(38.0)` and the capturing lambda produce the same count, and explain a lambda as a compiler-written functor.
- Run examples/06_stream_input.cpp: reading stops cleanly at 'abc', and the object is unchanged.

Instructor note: *The functor-lambda connection often produces an 'aha' moment: lambdas stop being magic once students see the class the compiler writes for them.*

5. Conversions and <=> (20 min)

- Run examples/07_conversion_operators.cpp. Uncomment the silent conversion to show explicit stopping it, and show explicit operator bool still working in if.
- Compile examples/08_spaceship.cpp with -std=c++20. Two defaulted lines give all six comparisons; then show a hand-written <=> and explain strong vs partial ordering.
- Close with the judgement rule: only overload an operator whose meaning is obvious for the type.

Common Student Misconceptions

Misconception	How to Address It
"friend breaks encapsulation, so never use it."	A friend declared inside the class, for operators that can't be members, is part of the class's interface. Misuse is making unrelated classes friends to avoid designing an interface.
"operator+ should return a reference, like +=."	+ creates a new value and must return by value; returning a reference to a local would dangle.
"One operator[] is enough."	Without a const version, indexing a const object or const& parameter won't compile.
"Every value type should have every operator."	Provide only operators whose meaning is obvious. Complex numbers, for example, have no natural ordering, so they should not have <.

Exercises & Module Project

Exercise: 30-operator-overloading-ii/exercises/exercise1.cpp - a Fraction value type kept in lowest terms with compound and symmetric binary operators, prefix/postfix ++, << and a failing >>, explicit operator double, and <=> (C++20). Project (three tracks): Generic - Money in Ghana cedis with a FeeLedger; EE - complex Impedance with a Network container and a deliberate decision NOT to provide ordering; Biomedical - Dose in micrograms with a MedicationChart. Each includes a functor used with an STL algorithm and a judgement table of operators provided and rejected.

Facilitation guidance: Read the judgement table before the code; the quality of the rejected operators and their reasons is the best indicator of understanding. Then test >> with bad input yourself, and check both versions of operator[] throw for bad keys or indices.

Discussion & Comprehension Check Questions

1. Why must operator<< and operator>> for your class be non-members?

Guidance: Their left operand is a stream, which you don't own and can't add members to. They are usually friends so they can reach private data.

2. Why is the postfix ++ given a dummy int parameter?

Guidance: It's the language's way of telling the two versions apart; the parameter is never used.

3. Why shouldn't an Impedance (complex number) class define operator<?

Guidance: Complex numbers have no natural order; any definition would be arbitrary and mislead readers. Use a named functor such as `ByMagnitude` when sorting is needed.

Assessment Ideas

- Students write `+=` and then `+` in terms of it for a new value type, plus a symmetric non-member `*`.
- Students list five operators for a given type and justify which to provide and which to reject.

Answer Key Notes

See `30-operator-overloading-ii/exercises/solutions/exercise1_solution.cpp` (compile with `-std=c++20`).

Wrap-Up & Bridge to Next Module

Bridge forward: "Our value types now behave naturally - including failing cleanly on bad input. Next module designs the error side of classes properly: exception hierarchies, throwing constructors, and the exception-safety guarantees."

Instructor Tips

- If time is short, protect `+=` then `+` (step 2) and functors (step 4).
- Confirm C++20 support on student machines before the session; Module 13's `==` and `<` are the fallback.